

Programovanie (3)

Skúška I.

Ivor Uhliarik

Fakulta matematiky, fyziky a informatiky
Univerzity Komenského v Bratislave

9. 1. 2025

Firmy a Startupy

Budete implementovať štruktúru firiem, ich zamestnancov a projektov.

Pred riešením úloh sa zoznámte s deklaráciami jednotlivých tried v hlavičkových súboroch.

Niektoré jednoduché metódy už boli implementované. V zdrojovom kóde sú pod komentárom

```
// Provided.
```

Úloha 1: Dátumy a projekty

Naprogramujte konštruktory a základné metódy štruktúr `Date` a `Project`.

- Konštruktor `Date` nastaví členské premenné podľa argumentov.
- V preťaženom operátore `Date::operator<<` vypíšte do streamu dátum vo formáte `"2025-01-31"`.
- Vytvorte konštruktor prázdneho projektu. Meno bude `"Empty project"` a dátum bude vynulovaný.
- Vytvorte konštruktor s parametrami a move konštruktor, ktorý presunie a vynuluje všetky členské premenné. Použite pritom volanie `move()` na typy, ktoré ho už implementované majú (`string`).
- Implementujte `Project::operator<<` podobne ako v `Date`. Očakávaný formát je `"<name> <date> <income> <soldFor>"`.

Úloha 2: Zamestnanci (I)

- Implementujte konštruktor prázdneho zamestnanca. Meno bude "Empty employee", plat 0 a pozícia Position::POS_NONE.
- Implementujte konštruktor s parametrami. Ak je meno prázdne, vyhod'te výnimku ReadException so správou "Name is empty". Ak je plat menší ako MIN_SALARY, vyhod'te výnimku ReadException so správou "Salary is lower than minimum".
- Správne implementujte move konštruktor. Pôvodnému objektu vyprázdniť či vynulujte premenné. Jeho pozíciu nastavte na POS_NONE.
- Implementujte setName a setSalary s rovnakým overovaním ako v konštruktoch s parametrami. V prípade výnimky členské premenné nemeňte.

Úloha 2: Zamestnanci (II)

- Implementujte `addActiveProject`. V zozname aktívnych projektov zamestnanca `Employee::projects` si uchovávame iba ich názvy. Metóda vráti `true` práve vtedy, keď sa projekt podarilo pridať.
- Implementujte metódu `clearProjects`, ktorá vyprázdni zoznam aktívnych projektov.

Úloha 3: Firmy (I)

- Implementujte konštruktor prázdnej firmy. Meno bude `"Empty company"` a dátum vynulovaný.
- Implementujte konštruktor s parametrami. Ak je meno prázdne, vyhodte výnimku `ReadException` so správou `"Name is empty"`.
- Implementujte obe verzie `addEmployee`. Verzia `addEmployee(Employee &&employee)` má použiť `move()` na presunutie zamestnanca, argument už nebudeme potrebovať.
- Implementujte obe verzie `addProject`. Verzia `addProject(Project &&project)` má použiť `move()` na presunutie projektu, argument už nebudeme potrebovať.
- Implementujte `move` konštruktor. Použite pritom volanie `move()` na typy, ktoré ho už implementované majú (`string`, `vector`).

Úloha 3: Firmy (II)

- Implementujte `findEmployeeByName`. Ak sa zamestnanec nenájde, vyhod'te `FindException` so správou `"There is no such employee"`.
- Implementujte `findProjectByName`. Ak sa projekt nenájde, vyhod'te `FindException` so správou `"There is no such project"`.

Projekt je predaný práve vtedy, keď je jeho hodnota `soldFor` nenulová.

- Implementujte `getMonthlyIncome`. Mesačný zisk firmy je suma všetkých mesačných ziskov `income` za projekty, ktoré neboli predané.
- Implementujte `getTotalIncome`. Celkový zisk firmy je suma všetkých ziskov za projekty: ak je projekt predaný, zarátajte jeho cenu, ináč jeho mesačný zisk.
- Implementujte `getMonthlyBalance` ako rozdiel mesačného zisku a sumy plátov zamestnancov.

Úloha 4: Ponuky

- Implementujte `removeEmployee`. Vráťte `true` práve vtedy, ak sa zamestnanec vymazal.
- Implementujte `transferEmployeeToCompany`. Zamestnanca z momentálnej firmy vymažte, odoberte mu projekty a pridajte ho do cieľovej firmy. Pomôžte si vytvorením kópie zamestnanca pred vymazaním.
- Implementujte `makeOfferFromCompany`. Metóda zohľadní momentálny plat zamestnanca v parametri a ponúkúkaný plat konkurenčnej firmy v parametroch. Ak je ponúkúkaný plat aspoň o 1000 vyšší ako jeho momentálny plat, zamestnanca presuňte do cieľovej firmy a vráťte `true`. V opačnom prípade vráťte `false`.
- Implementujte `Company::operator<` tak, aby porovnával podľa `getMonthlyBalance`, ale **opačne** (firma s vyššou hodnotou bude menšia).

Úloha 4: StartupList

- Implementujte `enlist` v triede `StartupList`. Jednoducho presuňte s pomocou operátora `move` firmu do zoznamu.
- Implementujte `sortByBalance`. Zoznam zoradíte podľa `getMonthlyBalance()` firiem zostupne.
- Implementujte `removeStartupsWithoutPotential`. Zo zoznamu vymažte všetky firmy, ktorých `monthlyBalance` je pod nulou. Ďalej zoradíte firmy podľa bilancie zostupne a vymažte všetky až na 20% najlepších firiem. Na určenie hranice si môžete pomôcť napr. aritmetikou s iterátormi (pripočítaním čísla k iterátoru). Ak spomenutých 20% nezodpovedá celému číslu, zaokrúhľujte nadol (ponechajte menej firiem).

Úloha 5: Načítavanie (I)

- Implementujte operátor >> pre každú z tried.
- Date používa formát "2025-01-31".
- Project používa formát
"<name> <date> <income> <soldFor>". Meno projektu je vždy jedno slovo.
- Employee používa formát
"<name> | <salary> <position> <number of projects> <proj1> <proj2> ..."
napr. "Elon Musk | 20000 CEO 2 SpaceX Tesla". Môžete uvažovať, že projekty zamestnancov sú zapísané správne (všetky existujú).

Úloha 5: Načítavanie (II)

- Company prečíta na prvom riadku meno firmy (vždy jedno slovo), dátum vzniku, počet projektov a počet zamestnancov. Nasleduje daný počet projektov a zamestnancov.

Google 1998-09-04 2 2

Search 1997-09-15 10000 50000

TensorFlow 2017-02-11 20000 100000

Larry Page | 15000 CEO 1 Search

Andrew Ng | 20000 DEVELOPER 2 Search TensorFlow

- Implementujte načítavanie zo súboru. Malo by byť priamočiare s použitím predchádzajúcich operátorov. Ak sa súbor nenájde, vyhod'te výnimku `ReadException` so správou `"Failed to read the file"`.